Reviewer Pack — Minimal Local Ring Norm in p-adic Geometry and Resolution Pr…

Entry bbed7fcad5b1 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 15:56:06 UTC

Minimal Local Ring Norm in p-adic Geometry and Resolution Proof Width

Entry ID: `bbed7fcad5b1` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-01 15:56:06 UTC

1. Conjecture

Field A (mathematical branch): p-adic Geometry **Field B** (complexity object): Resolution Proof Complexity

Statement:

> For every CNF φ with n variables, the minimal local ring norm (min_norm(φ)) of its associated structure in p-adic geometry is linearly correlated with its resolution proof width w(φ), such that min_norm(φ) = Θ(w(φ)).

Rationale (proposer's reasoning):

> p-adic geometry provides a non-Archimedean metric space that can capture the complexity of computation. The local ring norm measures the size of elements in this space, which may reflect the depth of the proof required to resolve the CNF.

Taxonomy category: `p-adic_geometry_resolution-proof_complexity` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `b79080d120efdc95`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if the computed correlation coefficient between min_norm(φ) and w(φ) for 30 random seeds is ≥ 0.8, with all individual seed correlations ≥ 0.7. The conjecture is falsified if any single seed's correlation coefficient is < 0.5 or the mean correlation coefficient across seeds is < 0.6.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	0.80	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `p-adic Geometry AND resolution proof complexity` - `min_norm(p-adic geometry) AND resolution proof width` - `θ-linear correlation between min_norm and w(φ) in p-adic geometry`

Top relevant hits considered: - [http://arxiv.org/abs/math-ph/0512018v2] On Phase Transitions for P-Adic Potts Model with Competing Interactions on a Cayley Tree - [http://arxiv.org/abs/2408.00810v3] p-adic Equiangular Lines and p-adic van Lint-Seidel Relative Bound - [http://arxiv.org/abs/hep-th/9410058v3] p-Adic description of Higgs mechanism I: p-Adic square root and p-adic light cone - [http://arxiv.org/abs/1503.08756v4] A bound on the norm of overconvergent p-adic multiple polylogarithms - [http://arxiv.org/abs/1701.07662v2] On unitarity of some representations of classical p-adic groups II - [http://arxiv.org/abs/2602.09898v1] p-adic symplectic geometry of integrable systems and Weierstrass-Williamson theory II - [http://arxiv.org/abs/2501.04451v2] Observation of the W-annihilation process $D_s^+ \to \omega\rho^+$ and measurement of $D_s^+ \to \varphi\rho^+$ in $D^+_s→π^+π+π^-π - [http://arxiv.org/abs/2309.02774v4] First Measurement of the Decay Asymmetry in the pure W-boson-exchange Decay $\Lambda_c^+ \to \Xi^0 K^+$

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def generate_cnf(n):
    cnf = []
    for _ in range(10 * n):  # Each variable appears in about 10 clauses
        clause = [random.choice([-1, 1]) * (i + 1) for i in random.sample(range(n), random.randint
        cnf.append(clause)
    return cnf

def min_local_ring_norm(cnf):
    p = 2  # Using a fixed p-adic field
    norm = 0
    for clause in cnf:
        val = sum(abs(lit) for lit in clause)
        if val > norm:
            norm = val
    return Fraction(norm, p**len(cnf))

def dpll_solve(cnf):
    def solve(variables, assignment):
```

```
        if not variables:
            return True
        var = variables[0]
        pos_var, neg_var = abs(var), -var
        if pos_var in assignment and assignment[pos_var] == False:
            return False
        if neg_var in assignment and assignment[neg_var] == True:
            return False
        assignment[var] = True
        if solve(variables[1:], assignment):
            return True
        assignment[var] = False
        assignment[-var] = True
        if solve(variables[1:], assignment):
            return True
        del assignment[var]
        del assignment[-var]
        return False

    variables = list(range(1, max(abs(lit) for lit in cnf) + 1))
    assignment = {}
    return solve(variables, assignment)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    correlations = []

    for n in n_values:
        cnf = generate_cnf(n)
        min_norm_val = min_local_ring_norm(cnf)
        width = dpll_solve(cnf)

        if width is None or min_norm_val is None:
            return {
                "metric_name": "correlation",
                "metric_value": 0.0,
                "instances_tested": len(n_values),
                "n_max": n,
                "conjecture_holds": False,
                "counterexample": "mapping_undefined"
            }

        correlation = min_norm_val * width
        correlations.append(correlation)

    mean_corr = sum(correlations) / len(correlations)
    std_corr = math.sqrt(sum((x - mean_corr) ** 2 for x in correlations) / len(correlations))

    return {
        "metric_name": "correlation",
        "metric_value": mean_corr,
        "instances_tested": len(n_values),
        "n_max": max(n_values),
        "conjecture_holds": all(0.7 <= corr >= 0.5 for corr in correlations) and mean_corr >= 0.8,
        "counterexample": ""
```

```
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_corr = sum(r["metric_value"] for r in results) / len(results)
    std_corr = math.sqrt(sum((r["metric_value"] - mean_corr) ** 2 for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_corr} std={std_corr} support_fraction={support_fracti
    elif any(r["metric_value"] < 0.5 for r in results) or support_fraction < 0.6:
        first_failing_seed = next(seed for seed, result in enumerate(results, start=seeds[0]) if r
        print(f"RESULT: FALSIFIED counterexample=\"first failing seed\" first_failing_seed={first_
    else:
        print(f"RESULT: INCONCLUSIVE reason=budget_exceeded n_tested={len(results)}")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a819b518.py", line 100, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a819b518.py", line 67, in run_trial
    width = dpll_solve(cnf)
            ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a819b518.py", line 55, in dpll_solve
    variables = list(range(1, max(abs(lit) for lit in cnf) + 1))
                             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a819b518.py", line 55, in <genexpr>
    variables = list(range(1, max(abs(lit) for lit in cnf) + 1))
                                  ^^^^^^^^
TypeError: bad operand type for abs(): 'list'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test code crashed before producing data, which means that the required correlation coefficients could not be computed to evaluate the conjecture. | next: Investigate and fix the error in the test code to compute the correlation coefficients and retest the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	28310
2	preregistration	ollama_remote	glm4:latest	0	0	11886
3	novelty	ollama_remote	glm4:latest	0	0	8594
4	novelty	ollama_remote	glm4:latest	0	0	12761
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20440
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15580
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20818
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12387
9	judge	ollama_remote	glm4:latest	0	0	8810

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 139585 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/bbed7fcad5b1.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/bbed7fcad5b1.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/bbed7fcad5b1.tar.gz` (if generated)